

BASES DE DATOS II

Trabajo Practico 3 - Parte I

Bases de Datos NoSQL con MongoDB

Seccion 1 - Bases de Datos NoSQL y Relacionales

Si bien las bases de datos NoSQL tienen diferencias fundamentales con los sistemas de bases de datos relacionales (RDBMS), algunos conceptos comunes pueden relacionarse. Responder las siguientes preguntas considerando MongoDB en particular como base de datos NoSQL.

1. ¿Cuales de los siguientes conceptos de RDBMS existen en MongoDB? En caso de no existir, ¿hay alguna alternativa? ¿Cual es?
 - Base de Datos
 - Tabla / Relacion
 - Fila / Tupla
 - Columna
2. ¿Existen claves foraneas en MongoDB? ¿Que diferencias existen con las bases de datos de tipo relacional?
3. Para acelerar las consultas, MongoDB tiene soporte para indices. ¿Que tipos de indices soporta?
4. En MongoDB existen dos tipos de vistas. Explicar brevemente cuales son y que diferencias existen entre ellas. Ademas, mencionar algunos casos donde podria utilizarlas.
5. Los documentos de una coleccion pueden diferir en la cantidad y tipos de campos. ¿Existen algunas formas de validar los elementos a insertar en una coleccion para evitar esta disparidad?
6. MongoDB tiene soporte para transacciones, pero no es igual que el de los RDBMS. ¿Cual es el alcance de una transaccion en MongoDB?
7. Las relaciones entre documentos en MongoDB pueden establecerse mediante documentos embebidos o referencias. Investigar como se implementa cada una y analizar las ventajas y desventajas de cada una, comparandola con la forma estandar de establecer relaciones en una base de datos relacional.
8. Tomando como referencia el modelo de los trabajos practicos anteriores y suponiendo que este podria mapearse a una base de datos en MongoDB, proponer algunos casos donde la relacion seria conveniente mapearla como referencia y otros como documentos embebidos. Justificar la eleccion.

Seccion 2 - Operaciones CRUD basicas

Descargar la ultima version de MongoDB desde el sitio oficial e ingresar al cliente de linea de comando para realizar los siguientes ejercicios.

2.1 Configuracion inicial e inserciones

9. Crear una nueva base de datos llamada "tours" y una coleccion llamada "recorridos".
10. En la nueva coleccion, utilizando el comando correspondiente, insertar el siguiente documento:

```
{ "nombre": "City Tour", "precio": 200, "stops": ["Diagonal Norte", "Avenida de Mayo", "Plaza del Congreso"], "totalKm": 5 }
```

11. Recuperar la informacion insertada usando `db.recorridos.find()` (puede agregarse `.pretty()` al final para ver los datos indentados). ¿Que diferencia se observa entre el documento insertado y el documento recuperado?
12. Agregar a la coleccion, utilizando un solo comando, los documentos especificados en el archivo "material_adicional_1.json" adjunto a esta practica.

2.2 Operaciones de actualizacion y eliminacion

Sobre la coleccion cargada en el punto anterior, realizar las siguientes operaciones:

13. Actualizar el recorrido "Cultural Odyssey" para que su total de kilometros sea 12.
14. Actualizar el listado de stops del recorrido "Delta Tour" para agregar "Tigre".
15. Aumentar un 10% el precio de todos los recorridos.
16. Eliminar el recorrido con nombre "Temporal Route".
17. Crear el array de etiquetas (tags) para la ruta "Urban Exploration" y agregar el elemento "Gastronomia" a dicho arreglo.

2.3 Consultas con find()

Sobre la coleccion generada anteriormente, realizar las siguientes consultas utilizando la funcion `find()`:

18. Obtener la ruta con nombre "Museum Tour".
19. Las rutas con precio superior a \$60.000.
20. Las rutas con precio superior a \$50.000 y con un total de kilometros mayor a 10.
21. Las rutas que incluyan el stop "San Telmo".

22. Las rutas que incluyan el stop "Recoleta" y no el stop "Plaza Italia".
23. El nombre y el total de km (si es que posee) de las rutas que incluyan el stop "Delta" y tengan un precio menor a \$50.000.
24. Las rutas que incluyen tanto "San Telmo" como "Recoleta" y "Avenida de Mayo" entre sus stops.
25. Solo el nombre de las rutas que dispongan de mas de 5 stops.
26. Las rutas que no tengan definido el total de sus kilometros.
27. Los nombres y el listado de stops de aquellas rutas que incluyen algun museo en sus recorridos.
28. La cantidad total de elementos que posee la coleccion.

Seccion 3 - Aggregation Framework

Aggregation Framework es una herramienta de MongoDB que permite ampliar la posibilidad de uso de su lenguaje de consulta. Posibilita realizar consultas complejas, filtrado, agrupacion y analisis de datos en tiempo real, aplicando una serie de etapas de procesamiento secuencialmente sobre los documentos de una coleccion.

3.1 Configuracion

29. Crear una nueva base de datos llamada "tours2". Guardar el archivo "generador1.js" adjunto a esta practica y ejecutarlo con:

```
load(<ruta del archivo 'generador1.js'>)
```

Si se utiliza un cliente que lo permita (por ejemplo Robo3T o MongoDB Compass), se puede ejecutar directamente en el espacio de consultas. Examinar las colecciones generadas antes de continuar.

3.2 Consultas con Aggregation Framework

Utilizando Aggregation Framework, realizar las siguientes operaciones:

Atencion Recordar examinar la estructura de las colecciones generadas por `generador1.js` antes de comenzar las consultas. Las rutas y stops en esta base de datos difieren en estructura de la coleccion de la Seccion 2.

30. Obtener una muestra de 5 rutas aleatorias de la coleccion.
31. Extender la consulta anterior para incluir en el resultado toda la informacion de cada una de las stops. Tener en cuenta que pueden ligarse por su codigo.
32. Obtener la informacion de las rutas (incluyendo la de sus stops) que tengan un precio mayor o igual a \$90.000.
33. Obtener la informacion de las rutas que tengan 5 stops o mas.
34. Obtener la informacion de las rutas que tengan incluido en su nombre el string "111".
35. Obtener solo las stops de la ruta con nombre "Route100".
36. Obtener la informacion del stop que mas apariciones tiene en rutas.
37. Obtener las rutas con precio inferior a \$15.000. Agregar a cada una una nueva propiedad que especifique la cantidad de stops que posee. Crear una nueva coleccion llamada "rutas_economicas" y almacenar estos elementos.
38. Por cada stop existente en la coleccion, calcular el precio promedio de las rutas que la incluyen.

Tip Si una consulta de aggregation comienza a ser dificil de leer, se pueden guardar las etapas individuales en variables JavaScript antes de armar el pipeline. Cada etapa no es mas que un objeto JSON.

Al completar esta practica, el alumno debera haber explorado los conceptos fundamentales de MongoDB, realizado operaciones CRUD sobre una coleccion real y construido pipelines de agregacion para consultas complejas.